08

SKT K-뉴딜 아카데미 · AI·자동화 기초

총평 · 보고서 틀 조건부 경고

오전의 요약 결과로 팀장이 아침에 읽을 보고서 한 장을 만듭니다.
숫자와 틀은 코드가, 총평 문장은 LLM(Large Language Model · 엘엘엠) 이 씁니다.

오후에는 세 교시에 걸쳐 보고서 한 장을 만듭니다

숫자와 틀은 코드가, 총평 문장은 LLM 이 씁니다.

총평 LLM → 보고서 틀 코드 → 조건부 경고 코드 → report_generator.py 하나로 묶음

교시	만드는 부분	LLM	산출물
5교시	요약을 다시 요약해 총평 두세 문장	부름	—
6교시	보고서 틀 · 숫자는 코드가 · 조건부 경고	부르지 않음	—
7교시	처음부터 끝까지 잇고 다른 날 경보로 다시	부름	report_generator.py · daily_report.md
8교시	오늘 만든 것 확인 · ALEPH(알레프) 기본 과제	—	—

5교시에 사용하는 용어는 네 개입니다

용어 (읽는 법)	뜻	5교시의 예
프롬프트 체이닝 (prompt chaining)	앞 단계의 답을 다음 단계의 질문에 넣는 방식	건별 요약 → 총평
총평 (executive summary · 경영진 요약)	보고서 한 장의 맨 앞 칸 — 밤사이 경보 전체를 두세 문장으로 정리한 글. 보고서 전체가 아님	`overview`
`.strip()` (스트립)	문자열 앞뒤의 공백 · 줄바꿈을 지움	`overview.strip()`
빈 문자열 `""`	글자가 하나도 없는 문자열 — LLM 호출이 실패했다는 뜻	`if text == "":`

보고서는 한눈에 보기 · 총평 · 건별 내역 세 칸입니다

```
# 야간 보안 관제 보고 (2026-10-07)

## 한눈에 보기
- 처리한 경보: 18건 (high 6건)
- **주의: high 경보 6건 – 건별 내역을 먼저 확인할 것**

## 총평
밤사이 관리자 계정을 노린 무차별 대입과 로그인 성공이 이어져 …

## 건별 내역 (위험한 것부터)
- [HIGH] E01 admin 계정에 로그인 실패 4회
```

## 한눈에 보기	건수와 경고 — 코드가 셈 (6교시)
## 총평	두세 문장 — LLM 이 씀 (5교시)
## 건별 내역	오전의 정렬 결과 — 코드가 씀 (6교시)

팀장이 18줄을 다 읽지 않아도 되게 보고서 맨 앞에 두세 문장 총평을 둡니다

총평은 보고서 한 장 안의 한 칸입니다. 그 칸만 읽어도 밤사이 무슨 일이 있었는지 압니다.

```
# 야간 보안 관제 보고 (2026-10-07)        ← 보고서 한 장
## 한눈에 보기                             ← 건수 · 경고 (코드가 셈)
## 총평                                    ← 두세 문장 (LLM 이 씀)
밤사이 관리자 계정을 노린 무차별 대입과 로그인 성공이 이어져 …
## 건별 내역                               ← 건별 요약 18줄
- [HIGH] E01 admin 계정에 로그인 실패 4회
- …
```

> **18줄을 다 읽을 시간이 없으니 두세 문장으로 줄인 총평을 맨 앞에 — 자세한 건 건별 내역에서.**

5교시에는 건별 요약 18줄로 총평을 받는 함수를 만듭니다

5교시가 끝나면 make_overview(건별 요약) 한 줄로 총평 두세 문장을 받습니다. 7교시에 보고서의 총평 칸에 넣습니다.

① 모으기 · 5-2
건별 요약 18줄을 문자열 하나로

→

② 묻기 · 5-3
LLM 에 보내 총평 두세 문장 받기

→

③ 실패 대비 · 5-4
못 받으면 「총평을 만들지 못했습니다」 넣기

→

④ 함수 · 5-5
①②③을 make_overview 하나로

③ 이 중요한 이유 — LLM 호출이 실패해도 보고서의 총평 칸이 비어 있지 않게, 실패했다는 사실을 문장으로 남깁니다.

프롬프트 체이닝 — 앞 단계의 답을 다음 질문에 넣습니다

앞 단계에서 LLM 이 준 답을, 다음 단계 질문에 그대로 넣어 다시 묻는 것입니다.

1단계 · 오전에 한 것
경보 원문 6건씩 → LLM

→

1단계의 답
건별 요약 18줄
E01 · high · admin 로그인 실패 4회 …

→

2단계 · 지금
그 18줄을 질문에 넣어 → LLM

→

2단계의 답
총평 두세 문장

경보 원문 18건을 한 번에 주고 총평까지 달라고 하면 질문이 길고 답이 흐려집니다.
이미 한 줄씩 요약한 것을 주면 LLM 은 전체를 묶는 일 하나만 하면 됩니다.

LLM 에 보내는 질문은 문자열 하나라서 리스트를 문자열 하나로 모읍니다

"지시문" + 리스트 는 **TypeError** — + 는 문자열끼리만 붙습니다.

```
summaries = [{"id": "E01", "summary": "로그인 실패"},
             {"id": "E02", "summary": "로그인 성공"}]
text = ""
for s in summaries:
    text = text + s["id"] + " " + s["summary"] + "\n"
print(len(text.split("\n")))      # 3
```

`text = text + ⋯ + "\n"`	한 줄씩 뒤에 붙임
`text.split("\n")`	줄바꿈 자리에서 나눠 리스트로. 마지막 `\n` 뒤에 빈 칸 하나가 생겨 두 줄인데 **3**

총평 질문에 넣으려고 18건을 summary_text 하나로 모읍니다

```
summary_text = ""
for s in sorted_summaries:
    summary_text = summary_text + f"- [{s['risk_level'].upper()}] {s['summary']}\n"
print(summary_text[0:60])
print("…", len(summary_text), "글자")
```

```
- [HIGH] admin 계정에 로그인 실패 4회
- [HIGH] 실패 직후 admin 로그인 성공 – 탈취 의심
… 507 글자
```

.upper()	위험도를 대문자로 — HIGH 가 눈에 띄게 (오전 4교시)
id 를 넣지 않음	총평에는 경보 번호가 필요 없음. 오전 문제 4-4 와 다른 점

총평 지시에 「숫자는 쓰지 마세요」 를 적어 보냅니다

LLM 은 건수를 세다 틀릴 수 있어서, 건수는 6교시에 코드가 세고 LLM 은 문장만 씁니다.

```
overview_instruction = ("다음은 밤사이 보안 경보 요약입니다. "
                        "팀장에게 보고할 총평을 한국어 두세 문장으로 쓰세요. "
                        "숫자는 쓰지 마세요. ""요약: ")
overview = call_llm(overview_instruction + summary_text)
overview = overview.strip()
```

두세 문장으로	출력 형식 — 길이를 정함 (10/6 3교시)
숫자는 쓰지 마세요	건수는 코드가 셈. LLM 이 센 숫자는 틀릴 수 있음
결과	숫자 없는 두세 문장. 실행할 때마다 문장은 조금 다름

보고서에 빈 줄이 생기지 않게 LLM 의 답 앞뒤를 `.strip()` 으로 다듬습니다

```
answer = "\n  관리자 계정 탈취가 의심됩니다.  \n"
print(len(answer), len(answer.strip()))     # 23 17
```

쓰는 것	결과
`answer`	앞뒤에 빈칸 · 줄바꿈이 붙어 있을 수 있음
`answer.strip()`	앞뒤만 지움. 문장 가운데 빈칸은 그대로

실패에 대비하려면 먼저 알아 둡니다 — `call_llm` 은 실패하면 빈 문자열을 돌려줍니다

```
overview = ""
print(overview == "", len(overview))     # True 0
```

값	== ""	뜻
""	True	아무 글자도 없음 — call_llm 이 실패하면 이 값
" "	False	빈칸 한 글자가 있음

> 한도 초과(429) 같은 실패 때 "" 가 옵니다. 그대로 넣으면 보고서의 총평 칸이 비어 버립니다.

총평을 못 받으면 칸을 비우지 않고 못 만들었다고 적습니다

```
overview = ""                                  # LLM 이 답을 못 준 경우
if overview == "":
    overview = "총평을 만들지 못했습니다. 건별 내역을 먼저 확인하세요."
print(overview)
```

```
총평을 만들지 못했습니다. 건별 내역을 먼저 확인하세요.
```

대신 쓸 문장	실패를 숨기지 않음. 보고서를 읽는 사람이 알아야 함
건별 내역을 먼저 확인하세요	총평이 없어도 다음에 할 일을 알려 줌

총평을 못 받으면 return 으로 바로 끝냅니다

if 안에서 return 을 만나면 함수가 끝나서 그 아래 줄은 실행되지 않습니다.

```
def check(text):
    if text == "":
        return "총평을 만들지 못했습니다"
    return text.strip()
print(check(""))
print(check(" 총평 "))
```

```
총평을 만들지 못했습니다
총평
```

들어온 값	실행되는 줄	돌려주는 값
""	첫 return 에서 함수가 끝남 — 아래 줄은 안 감	"총평을 만들지 못했습니다"
" 총평 "	if 를 건너뛰고 두 번째 return	"총평"

총평을 한 줄로 받으려고 make_overview 함수 하나로 묶습니다

```
def make_overview(sorted_summaries):
    summary_text = ""
    for s in sorted_summaries:
        summary_text = summary_text + f"- [{s['risk_level'].upper()}] {s['summary']}\n"
    text = call_llm(overview_instruction + summary_text)
    if text == "":
        return "총평을 만들지 못했습니다. 건별 내역을 먼저 확인하세요."
    return text.strip()
```

요약 글 만들기	반복 — 노트북에 미리 채워져 있음
call_llm · return 두 줄	실패 대비와 다듬기 — 직접 채우는 세 줄

5교시에 배운 것을 정리합니다

앞 단계의 답을 다음 질문에 넣는 것이 프롬프트 체이닝입니다.

하려는 일	쓰는 코드
요약을 글 하나로 모은다	`summary_text = summary_text + f"- [{…upper()}] {…}\n"`
총평을 받는다	`call_llm(overview_instruction + summary_text).strip()`
실패를 대비한다	`if text == "": return "총평을 만들지 못했습니다. …"`
묶는다	`make_overview(sorted_summaries)`

실습 ④ · 노트북 5교시 절 · 261007_pm_report_generator.ipynb

문제 5-1 ~ 5-5 · 도전 ⭐5-6

① **맨 위 위치 확인 셀과 준비 셀 셋을 먼저 실행합니다**
· 오전을 못 끝냈어도 여기서 시작

② **문제 5-1 · 5-2 · 5-4 — 모으기 · 실패 대비**
· LLM 을 부르지 않습니다

③ **문제 5-3 · 5-5 — 총평 받기 · make_overview**
· LLM 한 번씩. 429 면 1분 뒤

6교시에 사용하는 용어는 네 개입니다

용어 (읽는 법)	뜻	6교시의 예
템플릿 (template)	날마다 같은 모양으로 만드는 문서의 고정된 틀	제목 → 한눈에 보기 → 총평 → 건별 내역
조건부 경고	값이 기준을 넘을 때만 경고 문구를 넣는 것	high 3건 이상이면 **주의** 줄
`.startswith()` (스타츠위드)	문자열이 그 글자로 시작하면 `True`	`line.startswith("#")`
`f.read()` (리드)	파일 전체를 문자열 하나로 읽음	`text = f.read()`

보고서 전체를 LLM 에게 맡기면 모양과 건수가 날마다 달라집니다

틀과 숫자는 코드가, 총평 문장만 LLM 이 씁니다.

	LLM 이 보고서 전체를 씀	틀과 숫자는 코드, 총평만 LLM
소제목 · 순서	날마다 달라짐 — 어제와 비교할 수 없음	매일 같음
건수	「18건」이 「17건」이 될 수 있음	`len()` 으로 센 18건
경고 줄	LLM 의 판단에 따라 있다 없다 함	high 3건 이상이면 항상 들어감
총평 문장	LLM	LLM — 사람이 읽기 좋은 문장

6교시에는 보고서 틀을 코드로 만들어 저장합니다

① 건수 줄 **len · high** → ② 경고 줄 **3건 이상이면** → ③ 이어 붙이기 **네 부분** → ④ 저장 · 확인 **daily_report_… md**

순서	만드는 것	노트북	끝나면 보이는 값
① 건수 줄	전체 · high 건수를 코드로 셈	문제 6-2	- 처리한 경보: 18건 (high 6건)
② 경고 줄	high 3건 이상이면 경고	문제 6-3	- **주의: high 경보 6건 …**
③ 이어 붙이기	제목 · 한눈에 보기 · 총평 · 건별 내역	문제 6-4	# 야간 보안 관제 보고 (2026-10-07) …
④ 저장 · 확인	날짜가 든 파일로 저장하고 제목 줄만 출력	문제 6-5	제목 줄 네 개

보고서의 칸마다 건수와 순서는 코드, 총평만 LLM 이 쓰도록 정합니다

보고서의 칸	누가 쓰나	이유
건수 · high 개수	코드	세는 것은 틀리면 안 됨. LLM 은 숫자를 틀리게 셀 수 있음
총평 문장	LLM	사람이 읽기 좋은 문장은 LLM 이 잘 씀
제목 · 소제목 순서	코드	매일 같은 모양이어야 비교할 수 있음

5교시에 총평 지시에 「숫자는 쓰지 마세요」 라고 적은 이유입니다.
보고서에는 코드가 센 숫자만 들어갑니다.

건수 줄을 만들려고 전체는 len(),
high 는 .lower() 로 맞춰 셉니다

```
total = len(sorted_summaries)
high_count = 0
for s in sorted_summaries:
    if s["risk_level"].lower() == "high":
        high_count = high_count + 1
summary_line = f"- 처리한 경보: {total}건 (high {high_count}건)\n"
print(summary_line)
```

```
- 처리한 경보: 18건 (high 6건)
```

`len(sorted_summaries)`	전체 건수 — 18
`.lower() == "high"`	오전 4교시와 같음. 빠뜨리면 4건이 나옴

경고가 필요 없는 날을 위해 warning 을 빈 문자열로 먼저 만듭니다

```
high_count = 2
warning = ""

if high_count >= 3:
    warning = "주의\n"
print("[" + warning + "]")          # []
```

high_count	warning	보고서에서
3 이상	"- **주의: …**\n"	「한눈에 보기」 아래 경고 줄이 생김
2 이하	"" (빈 문자열)	이어 붙여도 아무것도 생기지 않음

high 가 3건 이상인 위험한 날은 경고 줄을 넣습니다

```
high_count = 6
warning = ""
if high_count >= 3:
    warning = f"- **주의: high 경보 {high_count}건 - 건별 내역을 먼저 확인**\n"
print(warning)
print("경고 줄 글자 수:", len(warning))
```

```
- **주의: high 경보 6건 - 건별 내역을 먼저 확인**
경고 줄 글자 수: 39
```

`>= 3`	**3 이상 — 3건이어도 경고가 들어감**
`**주의: ...**`	**마크다운 굵은 글씨 — 보고서에서 눈에 띄게**

날마다 나란히 비교하려고
네 부분의 순서를 코드에 고정합니다

순서	소제목	채우는 것
1	# 야간 보안 관제 보고 (날짜)	날짜 — 코드
2	## 한눈에 보기	건수 · 경고 — 코드
3	## 총평	문장 — LLM
4	## 건별 내역 (위험한 것부터)	정렬한 요약 — 코드

틀을 코드에 고정하면 어제와 오늘 보고서를 나란히 비교할 수 있습니다.

네 부분을 순서대로 이어 붙여 보고서를 만듭니다

```
report = f"# 야간 보안 관제 보고 ({today})\n\n"
report = report + "## 한눈에 보기\n" + summary_line + warning
report = report + "\n## 총평\n" + overview + "\n"
report = report + "\n## 건별 내역 (위험한 것부터)\n" + lines
```

summary_line · warning	앞의 두 장에서 코드가 만든 줄
overview	5교시의 총평 — LLM 이 쓴 문장. 여기서는 가정한 문장
소제목 앞 \n	위 부분과 빈 줄을 두려고 붙임

보고서에서 제목 줄만 고르려고 .startswith() 로 맨 앞 글자가 #인지 확인합니다

문자열.startswith("글자") 는 그 문자열이 그 글자로 시작하면 True 를 줍니다.

```
title = "# 야간 보안 관제 보고"
print(title.startswith("#"))              # True  - 맨 앞이 #
print(title.startswith("야간"))           # False - 맨 앞이 # 이라서

print("- E01 로그인 실패".startswith("#"))  # False - 맨 앞이 -
print("## 총평".startswith("#"))           # True  - ## 도 # 으로 시작
```

맨 앞만 봅니다
가운데에 # 이 있어도
상관없음

답은 True · False
그래서 if 뒤에 바로 씀

하나로 물으면
도 걸림 — 다음 장의 주의

만든 보고서에 소제목 세 칸이 빠짐없이 들어갔는지 # 으로 시작하는 줄만 골라 확인합니다

```
text = "# 제목\n- 내용\n## 소제목"
for line in text.split("\n"):
    if line.startswith("#"):
        print(line)
```

```
# 제목
## 소제목
```

`text.split("\n")`	줄마다 나눈 리스트
주의	`"## 소제목".startswith("#")` 도 **True**. 소제목만 고르려면 `"## "`

날마다 만든 보고서가 어제 것을 덮어쓰지 않게 파일 이름에 날짜를 넣어 저장합니다

```
today = "2026-10-07"
filename = f"daily_report_{today.replace('-', '')}.md"
with open(filename, "w", encoding="utf-8") as f:
    f.write(report)
print(filename)
```

```
daily_report_20261007.md
```

`today.replace("-", "")`	2026-10-07 → 20261007
`f"daily_report_{…}.md"`	날짜마다 다른 파일이 생김 — 어제 보고서를 덮어쓰지 않음
`"w"`	같은 날 다시 실행하면 그날 파일만 새로 씀

파일이 틀 그대로 저장됐는지 보려고 다시 읽어 제목 줄 네 개를 확인합니다

```
with open(filename, encoding="utf-8") as f:
    text = f.read()
for line in text.split("\n"):
    if line.startswith("#"):
        print(line)
```

```
# 야간 보안 관제 보고 (2026-10-07)
## 한눈에 보기
## 총평
## 건별 내역 (위험한 것부터)
```

f.read()	파일 전체를 문자열 하나로 읽음. 제목 줄 네 개가 나오면 틀이 그대로 저장된 것

6교시에 배운 것을 정리합니다

LLM 이 쓰는 것은 총평 하나뿐입니다.

보고서의 칸	누가	쓰는 코드
제목 · 소제목 순서	코드	`report = report + "## …\n" + …`
건수	코드	`len(sorted_summaries)` · `high_count`
경고 줄	코드	`if high_count >= 3: warning = …`
총평	LLM	`make_overview(…)`
저장	코드	`daily_report_{날짜}.md`

실습 ⑤ · 노트북 6교시 절 · 261007_pm_report_generator.ipynb

문제 6-1 ~ 6-5 · 도전 ⭐6-6

① **문제 6-1 ~ 6-3 — 경고 줄 · 건수 줄**
· 6-1 은 결과를 먼저 적습니다

② **문제 6-4 — 네 부분 이어 붙이기**
· 순서를 바꾸지 않습니다

③ **문제 6-5 — 저장하고 제목 줄 확인**
· 6교시는 LLM 을 부르지 않습니다

7교시에 사용하는 용어는 네 개입니다

용어 (읽는 법)	뜻	7교시의 예
파이프라인 (pipeline)	데이터를 여러 단계로 차례로 처리하는 구조	요약 → 정렬 → 총평 → 보고서
재현성	다른 날 데이터로 돌려도 같은 모양이 나오는 성질	10/7 · 10/8 보고서의 소제목이 같음
파일이름.함수	불러온 파일 안의 함수를 부르는 모양	`report_generator.build_report(…)`
문자열의 in	그 글자가 들어 있으면 True	`"**주의" in report`

내일은 경보만 바꿔 넣으면 보고서가 나오게
오늘 만든 함수를 .py 파일 두 개에 담아 둡니다

파일	담은 것	만든 때
event_summarizer.py	건별 요약 · 위험도순 정렬	오전 문제 4-5
report_generator.py	총평 · 보고서 틀 · 저장	지금 — 문제 7-2
내일 경보 6건	두 파일을 그대로 써서 보고서	문제 7-4 — 코드는 고치지 않음

학원 평가 기준이 여기서 확인됩니다 — 묶음 처리와 정렬 · 보고서 형식이 매번 같음 · high 3건 이상이면 경고.

7교시 — 내일 경보로도 같은 모양의 보고서가 나오는지 확인합니다

끝나면 `report_generator.py` 하나로 어느 날 경보든 보고서를 만들 수 있습니다.

① 파일 · 7-2
보고서 만드는 함수를 report_generator.py 에

→

② 오늘 · 7-3
10/7 경보 18건으로 보고서 저장

→

③ 내일 · 7-4
코드는 그대로, 10/8 경보 6건만 넣음

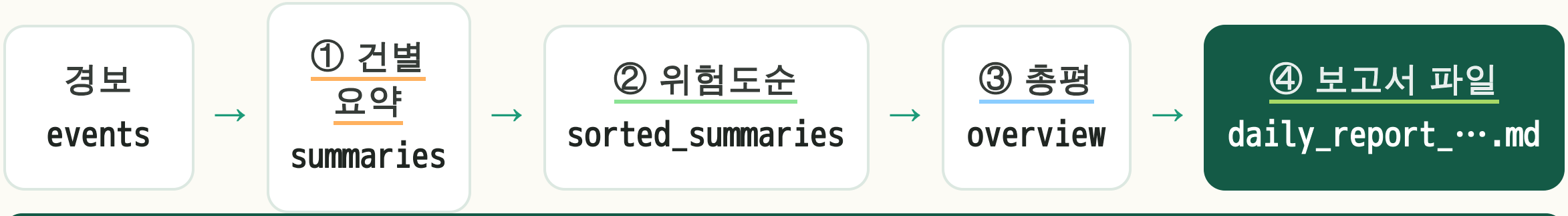
→

④ 비교 · 7-5
두 보고서의 소제목이 같은가

보고서는 날마다 같은 모양이어야 읽는 사람이 헷갈리지 않습니다. 날짜와 경보만 바꿔도 같은 틀이 나오면 성공입니다.
LLM 은 7-3 에서 한 번, 7-4 에서 두 번 부릅니다.

경보 파일을 넣으면 보고서 파일이 나오게 오늘 만든 함수 넷을 순서대로 잇습니다

이어 두면 매일 경보만 바꿔 넣고 이 다섯 줄만 실행하면 됩니다 — 이것이 파이프라인입니다.




```
summaries = event_summarizer.summarize_events(events)  # ① LLM
sorted_summaries = event_summarizer.sort_by_risk(summaries)  # ②
overview = report_generator.make_overview(sorted_summaries)  # ③ LLM
report = report_generator.build_report(sorted_summaries, overview, today)
report_generator.save_report(report, today)  # ④
```

한 번 돌리면 LLM 을 몇 번 부를까 — 건별 요약과 총평 두 단계입니다

단계	함수	파일	LLM
1	summarize_events	event_summarizer.py	3번 — 18건을 6건씩 세 묶음, 묶음마다 1번
2	sort_by_risk	event_summarizer.py	안 씀
3	make_overview	report_generator.py	1번
4	build_report → save_report	report_generator.py	안 씀

보고서의 틀을 함수에 넣어 두면 날짜만 바꿔도 매일 같은 모양이 나옵니다

7-4 에서 10/8 경보를 넣어도 보고서 모양이 같은 이유입니다. 작은 예로 제목 한 줄만 봅니다.

```
def make_title(today):
    return f"# 야간 보안 관제 보고 ({today})"
for day in ["2026-10-07", "2026-10-08"]:
    print(make_title(day))
```

```
# 야간 보안 관제 보고 (2026-10-07)
# 야간 보안 관제 보고 (2026-10-08)
```

함수 안의 틀(`# 야간 보안 관제 보고 (…)`)은 그대로이고, 넣은 날짜만 바뀝니다.
`build_report` 도 같아서, 경보가 바뀌어도 소제목 세 칸은 매일 같습니다.

파일에 담아 둔 함수를 다른 곳(노트북 · 다른 .py)에서 쓰려면

`파일이름.함수` 로 부릅니다

```
import json
json.dumps({"a": 1})                    # json 파일 안의 dumps 함수

import report_generator                 # ① report_generator.py 를 불러옴
report_generator.build_report(…)        # ② 그 파일 안의 build_report 를 부름
```

> 점(.) 앞은 파일 이름, 점 뒤는 그 파일 안의 함수 이름입니다.

report_generator.py 에 함수 넷을 담아 정리합니다

오후의 산출물입니다. 5 · 6교시에 쓴 줄을 함수로 묶어 파일 하나에 담습니다.

함수	하는 일	어디서 쓴 줄
make_lines(sorted_summaries)	- [HIGH] E01 요약 꼴 목록 줄	6교시 문제 6-4
make_overview(sorted_summaries)	총평을 받음. 실패하면 대신 쓸 문장	5교시 문제 5-5
build_report(…, overview, today)	빈칸 세 곳 — 세기 · 경고 · return	6교시 문제 6-2 ~ 6-4
save_report(report, today)	날짜가 든 파일로 저장하고 파일 이름을 돌려줌	6교시 문제 6-5

`build_report` 를 작은 데이터로 시험합니다 — high 가 2건이면 경고 줄이 없어야 맞습니다

경고 줄 **주의 는 high 가 3건 이상일 때만 붙습니다(6교시). 2건짜리로 시험해 경고가 빠지는지 봅니다.

```
import report_generator
print(report_generator.build_report(small, "총평 문장", "2026-10-08"))
```

```
# 야간 보안 관제 보고 (2026-10-08)

## 한눈에 보기
- 처리한 경보: 4건 (high 2건)

## 총평
```

오늘 건별 요약 18건으로 총평 → 보고서 → 저장 순서로 완성합니다

```
overview = report_generator.make_overview(sorted_summaries)  # LLM
report = report_generator.build_report(sorted_summaries, overview, today)
filename = report_generator.save_report(report, today)
print(filename, "를 저장했습니다")
```

```
daily_report_20261007.md 를 저장했습니다
```

순서	총평 → 보고서 → 저장
high 6건	3건 이상이라 경고 줄이 있음
"w"	6교시 6-5 의 파일을 이번에는 진짜 총평으로 덮어씀

보고서에 경고 줄이 들어 있는지 물으려고 "글자" in 문자열 을 씁니다

그 글자가 문자열 어디에든 들어 있으면 True 를 줍니다. .startswith() 는 맨 앞만 봅니다.

```
log = "admin 계정 로그인 실패"
print("admin" in log)      # True  - 맨 앞에 있음
print("로그인" in log)     # True  - 가운데 있어도 됨
print("root" in log)       # False - 없음
print("Admin" in log)      # False - 대문자 A 라서 다른 글자
```

.startswith("글자")
맨 앞만 봄

"글자" in 문자열
어디든 봄

같은 코드가 다른 날에도 되는지 보려고 **10/8 경보 6건**으로 처음부터 끝까지 돌립니다

```
today = "2026-10-08"
summaries = event_summarizer.summarize_events(events)  # LLM 1번
sorted_summaries = event_summarizer.sort_by_risk(summaries)
overview = report_generator.make_overview(sorted_summaries)  # LLM 1번 더
report = report_generator.build_report(sorted_summaries, overview, today)
report_generator.save_report(report, today)
print("경고 줄이 있나:", "**주의" in report)
```

```
경고 줄이 있나: False
```

events_1008.json	**10/8 경보 6건 — 코드는 하나도 고치지 않음**
False	**high 가 1건이라 경고가 빠짐. LLM 판단에 따라 다를 수 있음**

형식이 매번 같은지 소제목만 뽑아 비교합니다

소제목은 ##으로 시작하는지 if문으로 조건을 걸어줍니다.

```
for name in ["daily_report_20261007.md", "daily_report_20261008.md"]:
    headings = []
    with open(name, encoding="utf-8") as f:
        text = f.read()
    for line in text.split("\n"):
        if line.startswith("## "):
            headings.append(line)
    print(name, headings)
```

"## "	빈칸까지 붙여야 소제목만 고름 — 6교시
두 줄이 같음	경보 수 · 총평이 달라도 형식은 같음 — 학원 기준 「보고서 형식이 매번 같다」

경보가 18건이든 6건이든 보고서 형식은 매번 같습니다

팀장이 날마다 같은 자리에서 세 칸을 찾습니다 — 학원 기준 「보고서 형식이 매번 같다」

보고서	경보 수 · high	뽑은 소제목
daily_report_20261007.md	18건 · high 6건 — 경고 줄 있음	## 한눈에 보기 · ## 총평 · ## 건별 내역 (위험한 것부터)
daily_report_20261008.md	6건 · high 1건 — 경고 줄 없음	## 한눈에 보기 · ## 총평 · ## 건별 내역 (위험한 것부터)

경보 수 · 총평 · 경고 줄이 달라도 소제목은 같습니다.
학원 기준 「보고서 형식이 매번 같다」 를 코드로 확인했습니다.

7교시에 배운 것을 정리합니다

오후 산출물은 `report_generator.py` 와 `daily_report_20261007.md` 입니다.

단계	함수	파일
건별 요약	`summarize_events(events)`	`event_summarizer.py`
위험도순 정렬	`sort_by_risk(summaries)`	`event_summarizer.py`
총평	`make_overview(sorted_summaries)`	`report_generator.py`
조립 · 저장	`build_report(…)` · `save_report(…)`	`report_generator.py`

실습 ⑥ · 노트북 7교시 절 · 261007_pm_report_generator.ipynb

문제 7-1 ~ 7-5 · 도전 ⭐7-6

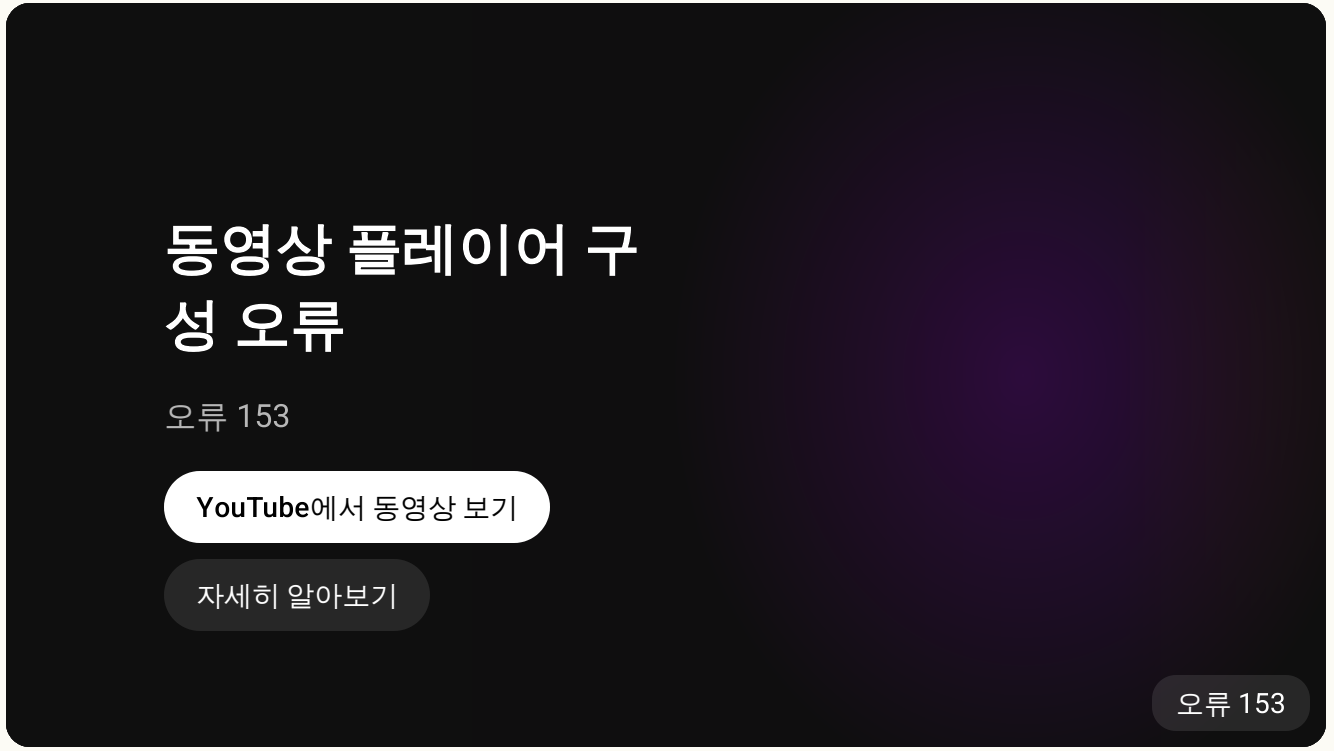
① **문제 7-1 ~ 7-2 — report_generator.py**
· 빈칸 세 곳. 고쳤으면 문제 셀 → 확인 셀 다시

② **문제 7-3 ~ 7-4 — 오늘 · 내일 보고서**
· LLM 한 번 · 두 번. 429 면 1분 뒤

③ **문제 7-5 — 두 보고서의 소제목 비교**
· LLM 을 부르지 않습니다

채용에서는 깃허브를 봅니다 — 첫 화면 README 꾸미기 (8분 11초)

계정 이름과 같은 저장소에 README 를 두면 프로필 첫 화면에 뜹니다.



8교시 — 오늘 만든 파일 넷이 있는지 확인합니다

새로 배우는 것은 없습니다. 없음 이 나오면 그 파일을 만드는 문제로 돌아갑니다.

노트북 8교시의 확인 셀을 실행합니다. 있음 **: 이 네 줄 나오면 됩니다.**

파일	만든 곳	하는 일
llm_client.py	10/6 오전	LLM 에 질문을 보내고 답을 읽음
event_summarizer.py	오전 문제 4-5	묶음으로 요약하고 위험도순으로 정렬
report_generator.py	오후 문제 7-2	총평을 받고 틀과 숫자를 코드로 채움
daily_report_20261007.md	오후 문제 7-3	오늘의 야간 보안 관제 보고 — 팀장이 읽는 파일

실습 ⑦ · 8교시 · 10분

좋은 README 만드는 법을 찾아 `docs/readme_guide.md` 로 정리합니다

알아볼 것	검색어
좋은 README 의 흐름	좋은 README 작성법 · README 템플릿 · awesome readme
README 마크다운 문법	GitHub README 마크다운 문법
커밋 메시지 앞머리	Conventional Commits · 커밋 메시지 컨벤션 feat fix
그림 넣기	GitHub mermaid 다이어그램 · mermaid flowchart 문법

자기 말로 정리하고 찾은 주소도 적습니다. 오늘은 저장만 합니다. 내일 아침 과제에서 `git push` 로 다른 파일과 함께 올라갑니다 — 오늘 웹 화면으로 올리지 않습니다.

커밋 메시지 앞에 `feat:` · `fix:` 를 붙이면 무엇을 바꾼 커밋인지 목록에서 바로 보입니다

앞머리	뜻	커밋 메시지 예
`feat:`	새 기능을 더함	`feat: add report_generator.py`
`fix:`	잘못된 것을 고침	`fix: count High as high`
`docs:`	문서만 고침 (README · docs/)	`docs: add README guide`
`refactor:`	동작은 그대로, 코드만 정리	`refactor: move prompt to config`
`test:`	확인 코드를 더하거나 고침	`test: add assert for build_report`
`chore:`	설정 · 잡일 (`.gitignore` 등)	`chore: ignore .venv`

Conventional Commits(컨벤셔널 커밋스 · 커밋 메시지 쓰는 약속)가 정한 것은 `feat` · `fix` 둘, 나머지는 널리 쓰는 관례입니다.

좋은 README 는 처음 온 사람이 위에서부터 읽으며 알게 합니다

순서	들어갈 것
① 제목 + 한 줄 소개	이 저장소가 무엇을 하는가
② 흐름 그림	머메이드 — 입력에서 결과까지
③ 폴더 구조	agent_core/ · network_zt/ · docs/
④ 실행하는 법	필요한 것 · 명령 · .env 에 키 이름만 (값은 절대 X)
⑤ 결과 예시	보고서 화면 일부
⑥ 기록	날짜별 docs/ 링크 · 다음에 할 것

```
# 깃허브가 그림으로 그려 줌
```mermaid
flowchart LR
  A[경보 파일] --> B[건별 요약]
  B --> C[총평] --> D[보고서 .md]
```
```

「무엇 → 생김새 → 돌리는 법 → 결과」 순서. 머메이드는 깃허브와 VS Code 미리 보기(1.121 이후)에서 그림으로 보입니다.

실습 ⑧ · 8교시 남은 시간

ALEPH 기본 과제 12개 풀기

각 과제에 투입할 수 있는 시간을 정하고, 그 안에 끝나도록 AI 를 활용합니다.
앞 과제가 승인돼야 다음 과제가 열립니다. 자기 차례의 과제부터 합니다.

과제	제목	계획 시간	과제	제목	계획 시간
T01	나를 소개하는 한 페이지	4~5h	T07	플랜두씨 다이어리 2 — 인증	6~8h
T02	내가 설계한 미니게임	6~7h	T08	내 소개 페이지에 패스키 달기	6~7h
T03	짤·카드 스튜디오	4~5h	T09	자동화 관제소	5~6h
T04	오늘의 진짜 정보판	6~7h	T10	AI와 함께 쓰는 첫 논문	8~10h
T05	대화가 끊겨도 이어지는 프로젝트	4~5h	T11	3만자 장편소설	6~8h
T06	플랜두씨 다이어리 1	8~10h	BR-A / B	마지막 과제 — 둘 중 하나	4~7h

내일은 오늘 만든 파일을 깃허브에 올리고 하나로 묶습니다

	오늘	내일
만든 것	파일 넷 — `report_generator.py` 등	오늘 파일을 아침 과제에서 깃허브에 올림
보고서	노트북에서 단계마다 실행	파이프라인을 하나로 묶어 한 번에 실행
오늘 할 일	깃허브에 올리지 않아도 됨	파일만 agent_core 에 있으면 됨

🔔 **17:30 — 수업을 마치고 오후 리추얼로 넘어갑니다. 리추얼은 20분이고 17:50 에 끝납니다.**